
Model-Based Control on CartPole with Learned Neural Network Dynamics and Model Predictive Control

Kyrillus Mehanni

Technical University of Vienna
e12028191@student.tuwien.ac.at

Feodor Razumovich

Technical University of Vienna
e12327933@student.tuwien.ac.at

Abstract

We study model-based reinforcement learning on the CartPole-v1 balancing task, following the approach of Nagabandi et al.: a small neural network is trained to predict the change in the environment state given the current state and action, and actions are selected with model predictive control (MPC) by sampling action sequences, rolling them out through the learned model, and executing the first action of the best sequence. The main question we wanted to answer is whether learning a dynamics model actually makes training more sample efficient than model-free RL. On this task it clearly does: after training on only 1000 random-policy transitions the controller already reaches a mean return of about 300 out of 500, more than ten times the random-policy return, and performance plateaus within about 5000 environment steps in total, while model-free methods such as DQN typically need on the order of 10^5 steps to solve CartPole. We ablate the planning horizon, the number of sampled action sequences, the sequence optimizer (random shooting vs. the cross-entropy method), the model output parameterization (state delta vs. absolute next state), and the model capacity and training data amount. Two findings stand out. First, predicting state deltas reduces the open-loop prediction error by a factor of 2 to 3 compared to predicting absolute states, even though the absolute variant has a lower normalized validation loss. Second, replacing the learned model with the true simulator dynamics does not improve control performance. The remaining gap to the maximum return is therefore not caused by model error, but by the planner itself: it only looks a short time ahead, and the survival reward gives it no reason to keep the cart near the center of the track.

<https://github.com/Kyrillus/Reinforcement-Learning-Model-Based-Control-on-CartPole>

1 Introduction

Model-free reinforcement learning methods such as deep Q-learning [Mnih et al., 2015] learn control policies directly from reward signals and can reach strong final performance, but they need a lot of data: even on small control problems, often tens or hundreds of thousands of environment interactions. In many settings, for example robotics, interactions with the real system are the dominant cost, which makes sample efficiency the central concern. Model-based reinforcement learning addresses this differently: instead of learning a policy, the agent learns a model of the environment dynamics from observed transitions and then uses that model for planning. Because the learned model can be queried arbitrarily often at no interaction cost, even a modest amount of real data can support a large amount of planning computation [Sutton and Barto, 2018, Deisenroth and Rasmussen, 2011].

In this project we follow the approach of Nagabandi et al. [2018] and apply it to the CartPole-v1 balancing task. A small multilayer perceptron (MLP) is trained with a supervised loss to predict the change in the environment state given the current state and action. Control is done with model

predictive control (MPC): at every time step the agent searches for an action sequence of fixed horizon H that maximizes the cumulative reward predicted by the model, executes only the first action of the best sequence, observes the true next state, and replans. As sequence optimizers we implement random shooting, which scores N uniformly sampled action sequences, and the cross-entropy method (CEM) [Rubinstein, 1999], which iteratively refits a sampling distribution to the best sequences it has seen. Following Nagabandi et al. [2018] and Ross et al. [2011] we alternate model training with on-policy data collection, so that the training distribution covers the states the MPC controller actually visits.

The goal of the project is not to push state-of-the-art numbers on CartPole, which is a solved problem, but to understand the behavior of the method through careful evaluation. Our contributions are:

- a compact, self-contained implementation of the full pipeline (data collection, dynamics model training, MPC planning, evaluation) in about 1300 lines of Python;
- a sample-efficiency evaluation showing that the method reaches strong performance within a few thousand environment steps;
- ablations over the planning horizon H , the number of sampled action sequences N , the sequence optimizer (random shooting vs. CEM), the model output parameterization (state delta vs. absolute next state), and the model capacity and training data amount;
- an oracle comparison in which the learned model is replaced by the true simulator dynamics, which lets us attribute the remaining performance gap to the planner rather than to model error.

2 Background

2.1 Problem setting

We consider a Markov decision process with states $s \in \mathcal{S}$, actions $a \in \mathcal{A}$, transition function $s_{t+1} = f(s_t, a_t)$, and reward function $r(s, a)$. The agent seeks to maximize the expected sum of rewards over an episode. In model-based RL the agent learns an approximation $\hat{f}_\theta$ of f from a dataset of observed transitions $\mathcal{D} = \{(s_t, a_t, s_{t+1})\}$ and uses it to select actions. We assume the reward function is known, which is reasonable here because the CartPole reward is a fixed and simple function of the state.

2.2 CartPole-v1

CartPole [Barto et al., 1983] is a classic control task in which a pole is attached by an unactuated joint to a cart that moves along a track. The state is four-dimensional: cart position x , cart velocity $\dot{x}$, pole angle θ , and pole angular velocity $\dot{\theta}$. The two discrete actions push the cart left or right with a fixed force. The reward is +1 for every step, and an episode terminates when the pole angle leaves $\pm 12^\circ$, the cart position leaves ± 2.4 , or after 500 steps, so the return equals the episode length and the maximum return is 500. We use the Gymnasium implementation [Brockman et al., 2016, Towers et al., 2024], which integrates the dynamics with explicit Euler steps of $\Delta t = 0.02$ s. Episodes start from a state where all four components are drawn uniformly from $[-0.05, 0.05]$. A uniformly random policy survives for roughly 20 steps, which gives a lower reference point for all results.

2.3 Model predictive control

MPC turns a dynamics model into a controller without ever representing an explicit policy. At state s_t the planner searches for the action sequence that looks best under the model over a fixed horizon H ,

$$(a_t^*, \dots, a_{t+H-1}^*) = \arg \max_{a_t, \dots, a_{t+H-1}} \sum_{k=0}^{H-1} r(\hat{s}_{t+k}, a_{t+k}), \quad \hat{s}_{t+k+1} = \hat{f}_\theta(\hat{s}_{t+k}, a_{t+k}), \quad (1)$$

with $\hat{s}_t = s_t$. Only the first action a_t^* is executed in the real environment; the resulting state is observed and the optimization is repeated. This receding-horizon scheme makes the controller robust

to model error, since the plan is corrected against reality at every step, but it also means the controller cannot see further than H steps ahead.

Solving Equation 1 exactly is infeasible (2^H sequences for binary actions), so we use sampling-based optimizers. *Random shooting* [Richards, 2005, Nagabandi et al., 2018] draws N action sequences uniformly at random, evaluates each by rolling it out through the model, and picks the best. The *cross-entropy method* [Rubinstein, 1999] maintains a sampling distribution over sequences, here an independent categorical distribution over the two actions per time step. In each of a few iterations it samples N sequences, selects the top fraction as elites, and refits the per-step distributions to the elite actions. This concentrates the search around promising sequences instead of spreading it uniformly, and it is also the optimizer used in later model-based RL work such as PETS [Chua et al., 2018].

3 Method

Our implementation follows Nagabandi et al. [2018], reduced to the parts needed for a discrete-action task.

3.1 Dynamics model

The dynamics model is an MLP $\hat{f}_\theta$ that takes the current state $s_t \in \mathbb{R}^4$ concatenated with a one-hot encoding of the action $a_t \in \{0, 1\}$ and outputs the predicted state *change*,

$$\hat{s}_{t+1} = s_t + \hat{\Delta}_\theta(s_t, a_t). \quad (2)$$

Predicting the delta rather than the absolute next state is the parameterization recommended by Nagabandi et al. [2018]: since consecutive states are similar, an absolute-state network must spend most of its capacity reproducing its input, and small relative errors in the output are large relative to the actual state change. We evaluate both parameterizations in Section 4.4.

The default network has two hidden layers of 64 units with ReLU activations, about 4900 parameters. Inputs and regression targets are standardized componentwise using the mean and standard deviation of the training set, which is important because the four state dimensions have quite different scales.

The model is trained with the mean squared error on the normalized targets using Adam [Kingma and Ba, 2015] with learning rate 10^{-3} and batch size 256. We hold out 10% of the transitions as a validation set, train for at most 200 epochs, stop early when the validation loss has not improved for 10 epochs, and keep the parameters of the best validation epoch.

3.2 Data collection and aggregation

Purely random exploration data is cheap but covers only the states a random policy reaches, which on CartPole means short trajectories that quickly tip over. A model trained only on such data is queried far outside its training distribution once the controller starts balancing successfully. Following Nagabandi et al. [2018], and in the spirit of DAgger [Ross et al., 2011], we therefore alternate between model training and on-policy data collection: the transitions visited by the MPC controller are added to the dataset and the model is retrained from scratch on the union. Algorithm 1 summarizes the loop; the episodes collected in each iteration double as the evaluation of the model trained in that iteration, which is what our learning curves report.

3.3 Planning

Both planners score a candidate action sequence by rolling it out through the model as a batch and summing the predicted rewards. Since the CartPole reward is simply +1 per surviving step, scoring reduces to counting how many steps of the imagined rollout stay within the termination thresholds; once an imagined rollout violates a threshold it stops accumulating reward. All N rollouts are propagated in parallel as a single batched forward pass per time step, so planning cost is H (times the number of CEM iterations) forward passes per environment step.

Random shooting uses the current state, samples N sequences uniformly, and returns the first action of the best one. CEM runs 5 iterations with N samples each and an elite fraction of 0.1, refitting an

Algorithm 1 Model-based training loop with on-policy aggregation

```
1:  $\mathcal{D} \leftarrow$  transitions from random-policy interaction
2: for iteration = 1, 2, ... do
3:   train  $\hat{f}_\theta$  on  $\mathcal{D}$  with a train/validation split and early stopping
4:   for a fixed number of episodes do
5:     run one episode with MPC (Equation 1) using  $\hat{f}_\theta$ ; record its return
6:     add the visited transitions to  $\mathcal{D}$ 
7:   end for
8: end for
```

Table 1: Default hyperparameters.

Dynamics model	MLP, 2 hidden layers $\times$ 64 units, ReLU, 4868 parameters
Model output	state delta Δs_t (one-hot action input)
Optimizer	Adam, learning rate 10^{-3} , batch size 256
Training	max. 200 epochs, 10% validation split, early stopping (patience 10)
Planning horizon	$H = 20$
Sampled sequences	$N = 200$ per iteration
CEM	5 iterations, elite fraction 0.1
Evaluation	15 episodes per configuration, fixed seeds, mean $\pm$ std

independent categorical distribution per planning time step to the elites, and returns the first action of the best elite of the final iteration. Unless stated otherwise both planners use horizon $H = 20$ and $N = 200$ sampled sequences per iteration.

4 Experiments

4.1 Setup

Implementation. The pipeline is implemented in Python with PyTorch [Paszke et al., 2019] for the dynamics model and Gymnasium [Towers et al., 2024] for the environment. All experiments run on a laptop CPU; the complete suite reported here (learning curves over three seeds plus all ablations) takes about 7 minutes. Table 1 lists the default hyperparameters, which are used everywhere unless a specific ablation varies them.

Datasets. All training data comes from the environment itself, so the dataset description is the collection procedure. For the learning-curve experiment (Section 4.2) each run starts from 1000 transitions collected by a uniformly random policy and then grows by three on-policy MPC episodes per iteration, following Algorithm 1. For the ablations we fix a single dataset so that all configurations see identical data: 2000 random-policy transitions plus on-policy MPC episodes, collected in rounds with the model retrained between rounds, until at least 8000 transitions are available, then subsampled uniformly to exactly 8000. Roughly three quarters of this dataset is on-policy data. Each transition is a tuple (s_t, a_t, s_{t+1}) with the four-dimensional state described in Section 2.2. Random-policy episodes are short (about 20 steps) and stay close to the initial state region, while the on-policy episodes cover the states a balancing controller visits, including slow drifts of the cart along the track.

Model training. Figure 1 shows the training and validation loss of the default model on the ablation dataset. The model reaches a validation MSE of about $5 \cdot 10^{-5}$ in normalized target space. Train and validation loss stay close throughout, so overfitting is not a concern at this data size; early stopping typically triggers after roughly 150 epochs.

Evaluation protocol. Every reported configuration is evaluated for 15 episodes with fixed, distinct environment seeds and planner seeds, and we report the mean and standard deviation of the episode returns. The standard deviations are large throughout (often 100 or more). This is a property of the task and controller rather than of the measurement: depending on the initial state, the cart sometimes

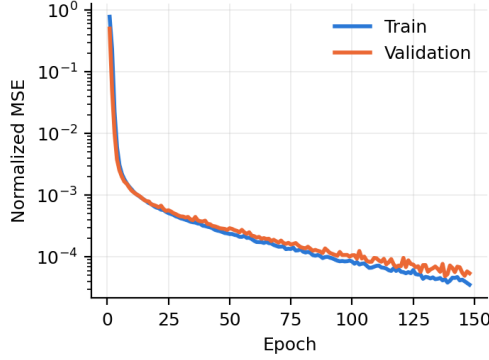


Figure 1: Training and validation loss (MSE on normalized targets) of the default dynamics model on the 8000-transition ablation dataset.

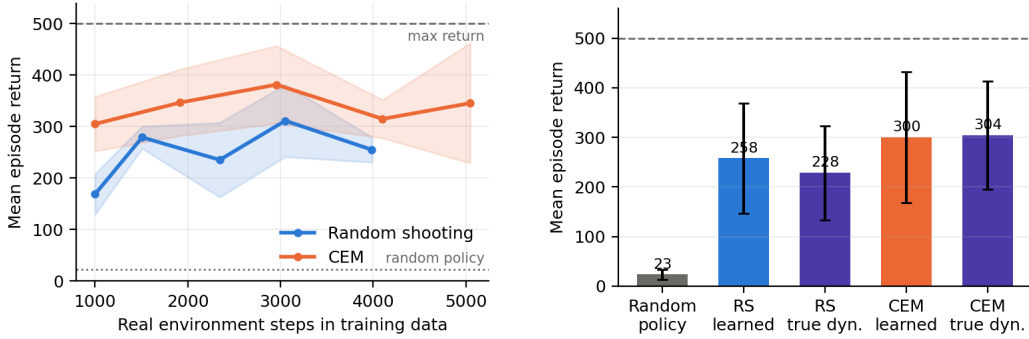


Figure 2: Left: mean episode return vs. real environment steps in the training data, for three seeds per planner (shaded: ± 1 std across seeds). Each point evaluates the model trained on the data collected up to that point. Right: baseline comparison at the default settings ($H = 20$, $N = 200$); “true dyn.” replaces the learned model with the exact simulator dynamics.

drifts slowly toward the track boundary and the episode ends early. We therefore base conclusions only on differences that are large relative to this spread and point out explicitly where observed differences are within the noise.

4.2 Overall performance and sample efficiency

Figure 2 (left) shows the primary metric, episode return as a function of real environment steps consumed for training data. The method needs very little data. The first model, trained only on 1000 random transitions, already yields a controller with a mean return around 170 (random shooting) to 300 (CEM), an order of magnitude above the random policy at 23. Performance improves modestly over the next iterations and plateaus around 250 for random shooting and 300 to 400 for CEM after roughly 5000 total environment steps, with substantial spread across seeds. For comparison, model-free methods such as DQN typically need on the order of 10^5 environment steps to solve CartPole [Mnih et al., 2015], two orders of magnitude more.

The plateau below the maximum return of 500 is the more interesting observation, and Table 2 explains it. Replacing the learned model with the exact simulator dynamics changes performance by less than the evaluation noise in both directions (random shooting even drops slightly). In other words, at the default settings the learned model is already good enough that the controller would not benefit from a perfect one; the bottleneck is the planner itself. We return to this in Section 4.5.

Table 2: Baseline comparison at the default settings, 15 episodes per entry.

	Learned model	True dynamics
Random policy	23.3 ± 9.5	
MPC, random shooting	257.7 ± 111.1	228.4 ± 94.9
MPC, CEM	300.3 ± 131.7	304.1 ± 109.0

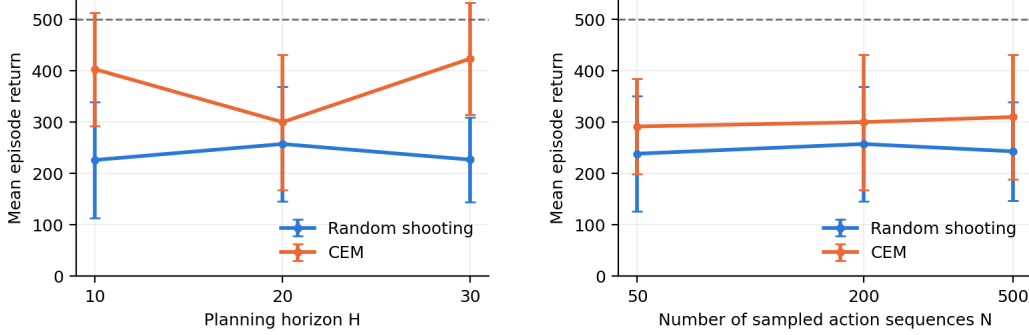


Figure 3: Planner ablations with the default learned model (error bars: ± 1 std over 15 episodes). Left: planning horizon H at $N = 200$. Right: number of sampled sequences N at $H = 20$ (log scale).

4.3 Planner ablations

Random shooting vs. CEM. Across every configuration in Figure 3, CEM matches or beats random shooting, by roughly 40 to 180 return depending on the setting. Both planners evaluate the same number of model rollouts per iteration; CEM just uses them better, because later samples concentrate on action patterns that already worked in earlier iterations. This matters because a uniformly random action sequence of length 20 or 30 usually topples the pole in the model rollout, so the best of N uniform samples is a noisy choice, whereas CEM’s refitted per-step distributions keep whole surviving sequences in the pool.

Planning horizon. Random shooting is flat in H within noise (226.6 ± 113.3 , 257.7 ± 111.1 , 227.3 ± 82.9 for $H = 10, 20, 30$). The CEM sweep in Figure 3 suggests a dip at $H = 20$ (403.3 , 300.3 , 424.0), but this does not survive a closer look: repeating the CEM sweep with a different set of evaluation seeds gives 375.9 ± 118.2 , 344.5 ± 115.2 , and 348.5 ± 126.5 , so both the dip at $H = 20$ and the peak at $H = 30$ are within the evaluation noise. Our conclusion is that over the tested range performance is largely insensitive to the horizon for both planners; the reliable effect in this figure is the planner gap, not the horizon. A likely reason for this insensitivity is discussed in Section 4.5.

Number of sequences. Increasing N from 50 to 500 changes both planners by less than one standard deviation (random shooting $238.9 \rightarrow 243.4$, CEM $291.9 \rightarrow 310.3$). With only two actions, 50 samples per iteration already seem to cover the sequences that matter; compute is better spent on CEM iterations or the horizon than on more samples.

4.4 Model ablations

Delta vs. absolute next state. Figure 4 compares predicting the state change Δs_t against predicting s_{t+1} directly, with both networks trained identically on the same data. In episode return the difference is visible but within the noise (257.7 ± 111.1 vs. 235.3 ± 94.5). The open-loop rollouts show the real difference: the delta model’s prediction error is a factor 2 to 3 lower at every unroll depth, e.g. 0.060 vs. 0.161 after 30 steps. Interestingly, the absolute model has the *lower* validation loss ($8 \cdot 10^{-6}$ vs. $4.9 \cdot 10^{-5}$), but this comes from the normalization: its targets s_{t+1} vary over the whole state range, so a small normalized error still corresponds to a large error relative to the actual per-step state change. This is exactly why Nagabandi et al. [2018] recommend predicting deltas,

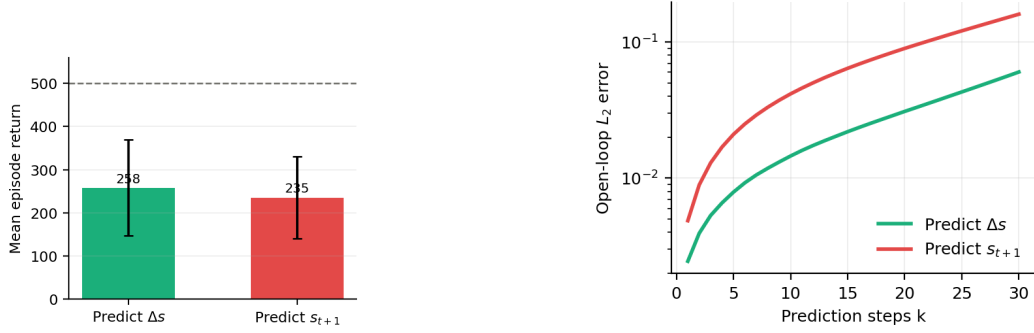


Figure 4: Output parameterization ablation, both models trained on the same 8000 transitions. Left: episode return with random shooting at default settings. Right: open-loop prediction error, i.e. mean L_2 distance between the predicted and true state after k steps when unrolling the model on recorded action sequences from held-out trajectories (log scale).

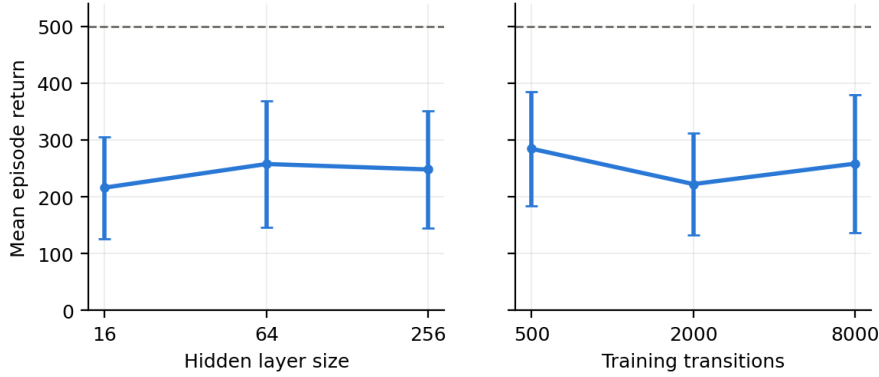


Figure 5: Model capacity and data ablations, random shooting at default settings (error bars: ± 1 std over 15 episodes). Left: hidden layer width, trained on 8000 transitions. Right: number of training transitions, width 64.

and it also shows that validation losses cannot be compared between the two output types. That the return gap stays small despite the accuracy gap is consistent with the oracle result: at $H = 20$ the planner does not need a highly accurate model.

Model capacity and data amount. Figure 5 and Table 3 vary the hidden layer width (16, 64, 256 units) at the full 8000 transitions, and the number of training transitions (500, 2000, 8000) at the default width. Neither axis produces differences beyond the noise. A network with 16 hidden units and only 452 parameters, trained on 500 transitions, is essentially as good for control as the largest configuration, even though the validation MSE improves by a factor of about 75 between 500 and 8000 transitions. CartPole dynamics are smooth and low-dimensional, so very little capacity and data suffice to reach the model quality the planner can exploit; past that point, better models stop paying off. We expect this to look different on higher-dimensional tasks, where Nagabandi et al. [2018] report clear gains from more data.

4.5 Discussion

What works. The overall recipe works well for how simple it is: a 5000-parameter MLP, a few thousand transitions, and a sampling-based planner solve the task to a large extent within minutes of CPU time and with two orders of magnitude fewer environment steps than typical model-free baselines. The delta parameterization and input/target normalization matter for model quality and cost nothing. CEM is a clear improvement over random shooting at equal sample budget, and it is the only knob among those we ablated whose effect is clearly outside the evaluation noise.

Table 3: Model ablations: episode return and validation MSE (normalized targets).

Configuration	Parameters	Return	Val. MSE
Hidden width 16	452	216.2 ± 89.8	$2.7 \cdot 10^{-5}$
Hidden width 64 (default)	4868	257.7 ± 111.1	$4.9 \cdot 10^{-5}$
Hidden width 256	68 612	248.1 ± 103.4	$6.2 \cdot 10^{-5}$
500 transitions	4868	284.5 ± 100.6	$1.1 \cdot 10^{-3}$
2000 transitions	4868	222.4 ± 90.1	$4.4 \cdot 10^{-4}$
8000 transitions (default)	4868	258.4 ± 121.1	$1.5 \cdot 10^{-5}$

Where it struggles. No configuration reliably reaches the 500-step cap, and the evaluation variance is high everywhere. The oracle experiment shows where the problem is: it is not model error but the combination of a short planning horizon with a reward that says nothing about *how well* the system is doing. Every action sequence that keeps the pole up for all of the horizon gets the same maximal score, so among such sequences the planner picks arbitrarily, and nothing penalizes a slow drift of the cart toward the edge of the track that only becomes fatal 50 steps later. The failure statistics match this: over 30 evaluation episodes at the default settings, 13 ended with the cart reaching the track boundary, 13 with the pole falling, and only 4 reached the 500-step cap, so half of the failures are the slow kind that a 20-step lookahead only notices when it is too late. Extending the horizon within the tested range does not fix this (Section 4.3): even $H = 30$ is only 0.6 s of lookahead, and as long as many sequences survive the whole horizon the objective still cannot distinguish a centered cart from one drifting outward. A perfect model does not help either, which rules out the model as the cause.

Limitations of the evaluation. With 15 episodes per configuration and per-episode standard deviations above 100, only the large effects (vs. the random policy, CEM vs. random shooting, the open-loop error gap) are clearly outside the noise; the smaller ordering effects between adjacent configurations should not be over-interpreted. The standard errors of the reported means are roughly 25 to 35 return.

5 Conclusion

We implemented and evaluated the model-based RL approach of Nagabandi et al. [2018] on CartPole-v1: an MLP dynamics model trained on observed transitions, combined with sampling-based MPC and on-policy data aggregation. The method reaches roughly ten times the random-policy return after a single model fit on 1000 random transitions and plateaus at a mean return of roughly 250 (random shooting) to 350 (CEM) within about 5000 environment steps, far more sample efficient than typical model-free learners on this task. The ablations give a consistent overall picture. On the model side the problem is easy: a few hundred parameters and a few hundred transitions produce a model whose remaining error no longer limits control, and predicting state deltas rather than absolute states improves open-loop accuracy by a factor of 2 to 3 at no extra cost. On the planner side the problem is harder than it looks: with a pure survival reward and a short horizon, the controller does not react to problems that build up slowly, and this, not model quality, is what keeps every configuration below the maximum return. For tasks of this scale, the practical lesson is to spend effort on the sequence optimizer and the planning objective rather than on model capacity or extra data. A second lesson is to validate dynamics models with open-loop rollouts; the one-step validation loss can be misleading, as the delta vs. absolute comparison showed.

6 Future work

The clearest next step follows directly from the failure analysis in Section 4.5: give the planner a reason to keep the cart centered. The simplest version is to add a small penalty on $|x|$ and $|\theta|$ to the planning objective, which would break the ties between sequences that all survive the horizon. Since the drift toward the track boundary is responsible for about half of the failures, we would expect this change alone to push many more episodes to the 500-step cap, and it is the first thing we would try with more time. A more general version of the same idea is to learn a value function for the state at

the end of the horizon, so that the planner gets an estimate of what happens beyond the H steps it can see. Finally, Nagabandi et al. [2018] continue by distilling the MPC controller into an explicit policy that is then fine-tuned with model-free RL; this would remove the planning cost at execution time and is the natural continuation of the approach we followed.

Division of work

The work was split roughly along the module boundaries of the code base. Feodor Razumovich implemented the core pipeline: the dynamics model and its training setup (`dynamics/`), the data collection and aggregation (`data/`), and the MPC planners (`mpc/`). Kyrillus Mehanni implemented the evaluation harness, the ablation suite, and the figure generation (`evaluation/`, `ablation/`, `main.py`) and ran the experiments. The split was not strict; both of us worked on and debugged both parts, and the analysis and the report were written together.

Use of AI assistance

Table 5 (Anthropic) was used during the project to help with understanding the reference papers, refactoring code, and debugging.

References

- Andrew G. Barto, Richard S. Sutton, and Charles W. Anderson. Neuronlike adaptive elements that can solve difficult learning control problems. *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-13(5):834–846, 1983.
- Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. OpenAI Gym. *arXiv preprint arXiv:1606.01540*, 2016.
- Kurtland Chua, Roberto Calandra, Rowan McAllister, and Sergey Levine. Deep reinforcement learning in a handful of trials using probabilistic dynamics models. In *Advances in Neural Information Processing Systems*, 2018.
- Marc Peter Deisenroth and Carl Edward Rasmussen. PILCO: A model-based and data-efficient approach to policy search. In *International Conference on Machine Learning*, 2011.
- Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.
- Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Belle-mare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.
- Anusha Nagabandi, Gregory Kahn, Ronald S. Fearing, and Sergey Levine. Neural network dynamics for model-based deep reinforcement learning with model-free fine-tuning. *IEEE International Conference on Robotics and Automation (ICRA)*, 2018. arXiv:1708.02596.
- Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, 2019.
- Arthur George Richards. Robust constrained model predictive control. *PhD thesis, Massachusetts Institute of Technology*, 2005.
- Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *International Conference on Artificial Intelligence and Statistics*, 2011.
- Reuven Y. Rubinstein. The cross-entropy method for combinatorial and continuous optimization. *Methodology and Computing in Applied Probability*, 1(2):127–190, 1999.

Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.

Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, et al. Gymnasium: A standard interface for reinforcement learning environments, 2024. [arXiv:2407.17032](https://arxiv.org/abs/2407.17032).